

Type annotation syntax

Expression	Explanation
<code>x: T</code>	Variable <code>x</code> has type <code>T</code>
<code>def f (x: T) -> U</code>	Function <code>f</code> takes a parameter <code>x</code> of type <code>T</code> and returns a value of type <code>U</code>
<code>T U</code>	Either type <code>T</code> or type <code>U</code>
<code>list[T]</code>	A list of items of type <code>T</code>
<code>dict[T, U]</code>	A dictionary with keys of type <code>T</code> and values of type <code>U</code>
<code>Iterator[T]</code>	An iterator over items of type <code>T</code>

HTML Library API

```
# An HTML element
class Elem:
    # Name of the tag of this element
    tag: str

    # Attributes of this element
    attrs: dict[str, str]

    # Child elements of this element can be sub-elements or text
    elems: list[Elem | str]

    # Initialisation method
    def __init__(
        self,
        tag: str,
        attrs: dict[str, str] = {},
        elems: list[Elem | str] = []
    )

    # Equality check method
    # Returns True if `other` is an Elem which is equal to `self`
    # Recurses through all child elements
    def __eq__(self, other) -> bool

    # Add a child element
    def add_elem(self, elem: Elem) -> None

# A complete HTML document
class Document:
    # Optional text that comes before the root element
    prolog: str | None

    # The root element of the document tree
    root: Elem

    # Equality check method
    # Returns True if `other` is a Document which is equal to `self`
    # Recursively checks entire document tree
    def __eq__(self, other) -> bool

    # Return an iterator over all elements with the given tag name
    # Recurses through entire document tree
    def iter_tag(self, tag: str) -> Iterator[Elem]
```

```
# Return an iterator over all elements with the given attribute
# Recurses through entire document tree
def iter_attr(self, key: str, value: str) -> Iterator[Elem]

# Return an iterator over all text content in the document
# Recurses through entire document tree
def iter_text(self) -> Iterator[str]

# Read a file and parse it as HTML
# Raises IOError if file cannot be read
# Raises ParseError if file contents cannot be parsed as HTML
def parse_file(filepath: str) -> Document

# Parse a string as HTML
# Raises ParseError if string cannot be parsed as HTML
def parse_string(raw: str) -> Elem
```